1. Problem Title & Source

Title: **Drum Beat Student Rearrangement**

Source: **TCS NQT Previous Coding Questions**

2. Problem Statement (Clear Version)

In a school activity, **N students are standing in a circle**.

A teacher beats a drum **M times**. Each drum beat causes the students to **shift their positions in the circle**.

The rule is:

- After each drum beat, **the first student moves to the end of the line**.
- This process repeats **M times**.

Your task is to determine **the final arrangement of students after M drum beats**.

In simple terms:

- Students are arranged in a list.
- Each drum beat performs a **left rotation of the list by one position**.

3. Examples

Example 1

Input

5

1 2 3 4 5

2

Output

3 4 5 1 2

Explanation

Initial arrangement

1 2 3 4 5

After first drum beat

2 3 4 5 1

After second drum beat

3 4 5 1 2

Final arrangement

3 4 5 1 2

Example 2

Input

4

10 20 30 40

3

Output

40 10 20 30

Explanation

Initial

10 20 30 40

Rotations

1 → 20 30 40 10

2 → 30 40 10 20

3 → 40 10 20 30

Final arrangement

40 10 20 30

4. Constraints

1 ≤ N ≤ 100

1 ≤ M ≤ 10^6

Observations

- Repeating rotation **M times directly is inefficient**
- Rotation pattern repeats every **N shifts**

5. Core Concept (Pattern)

Array Rotation

Important idea:

Effective rotations = M % N

6. Thought Process

Brute Force Idea

Rotate array **M times**.

Example

```
for i in range(M)
   rotate array
```

Time complexity

O(N × M)

This becomes slow if **M is very large**.

Optimized Idea

Observe that after **N rotations**, the array returns to original.

Therefore

effectiveRotation = M % N

Then rotate only **effectiveRotation times**.

7. Visualization

Example

N = 5

M = 7

Effective rotation

7 % 5 = 2

So only **2 rotations needed**.

8. Pseudocode

```
read N
read array
read M

k = M % N

for i from k to N-1
   print arr[i]

for i from 0 to k-1
   print arr[i]
```

9. Code Implementation

Python

```
n = int(input())
arr = list(map(int,input().split()))
m = int(input())

k = m % n

result = arr[k:] + arr[:k]

print(*result)
```

Java

```
import java.util.*;

class Main {

   public static void main(String[] args){

      Scanner sc = new Scanner(System.in);

      int n = sc.nextInt();

      int[] arr = new int[n];

      for(int i=0;i<n;i++)
         arr[i] = sc.nextInt();

      int m = sc.nextInt();

      int k = m % n;

      for(int i=k;i<n;i++)
         System.out.print(arr[i] + " ");

      for(int i=0;i<k;i++)
         System.out.print(arr[i] + " ");
   }
}
```

10. Time & Space Complexity

Time Complexity

O(N)

Only one traversal of array.

Space Complexity

O(1)

(If printing directly)

11. Common Mistakes

1. Rotating array **M times**
2. Forgetting modulo optimization
3. Incorrect rotation direction
4. Using extra loops unnecessarily

12. Detailed Dry Run

Input

N = 5

arr = [1,2,3,4,5]

M = 2

Effective rotations

2 % 5 = 2

Rotation

3 4 5 1 2

Output

3 4 5 1 2

13. Real-Life Applications

Array rotation appears in:

- Circular buffers
- Task scheduling
- Round-robin algorithms
- CPU process scheduling

14. Common Traps

1. Misinterpreting left vs right rotation
2. Forgetting modulo optimization
3. Off-by-one errors
4. Incorrect slicing logic

15. Related Problems

Array rotation problems:

LC 189 Rotate Array

LC 61 Rotate List

LC 396 Rotate Function

16. Alternate Approaches

Approach	Complexity	Notes
Repeated rotation	O(N×M)	inefficient
Modulo + slicing	O(N)	optimal

17. Why This Solution Works

Because rotating an array **N times returns it to its original state**, any number of rotations greater than N can be reduced using modulo.

Thus we only perform the **necessary effective rotations**.

18. Variations / Follow-Up Questions

Possible interview variations:

1. Rotate array **to the right**
2. Rotate array **K positions**
3. Circular queue implementation
4. Rotate matrix rows